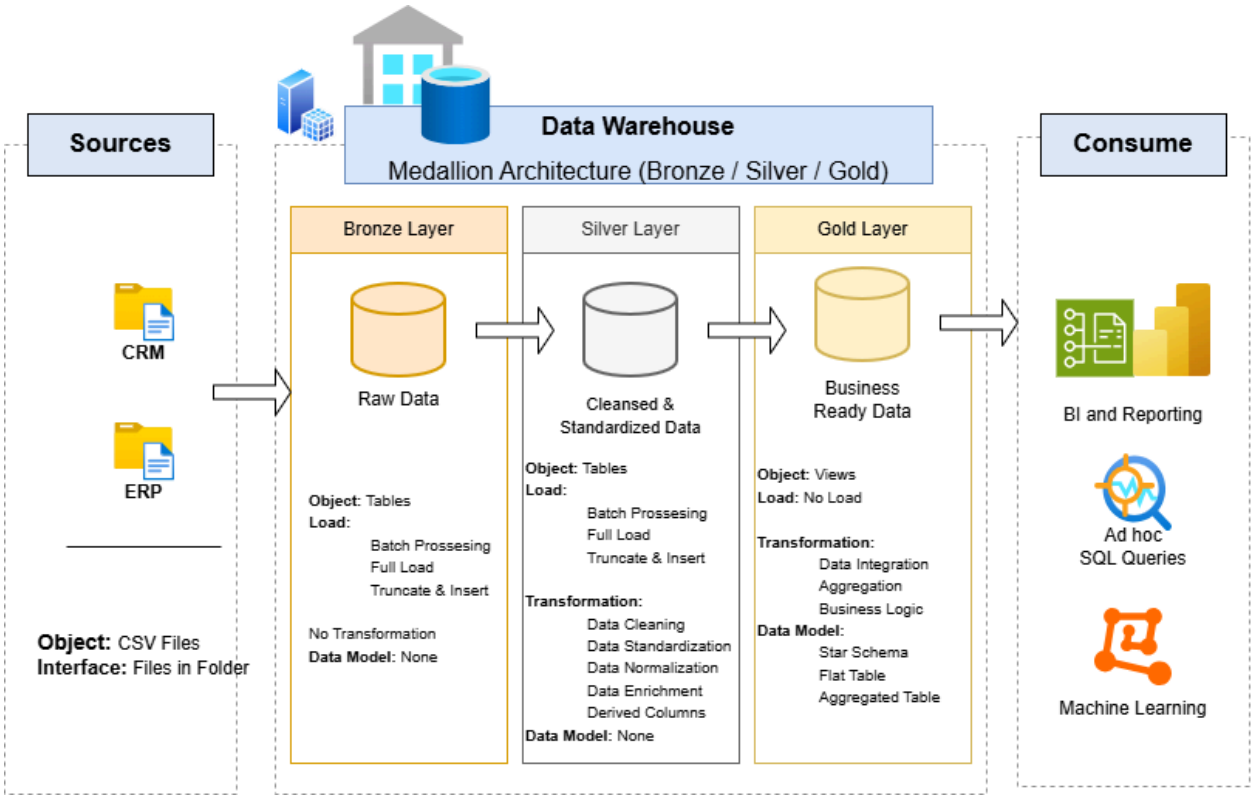
Project Documentation

Detailed documentation for the SQL Server data warehouse project. For a short overview, see the README.

> **Credit:** This project was built by following the Data with Baraa SQL data warehouse tutorial. The source datasets come from his course materials and are used under the MIT license.

---

1. Architecture



The warehouse follows the Medallion architecture with three layers:

Layer	Purpose	Object type	Load method
Bronze	Raw data exactly as it comes from the source CSV files	Tables	Full load: truncate and insert
Silver	Cleaned, standardized and enriched data	Tables	Full load: truncate and insert
Gold	Business-ready data modeled as a star schema	Views	No load (views read from Silver)

---

2. Data sources

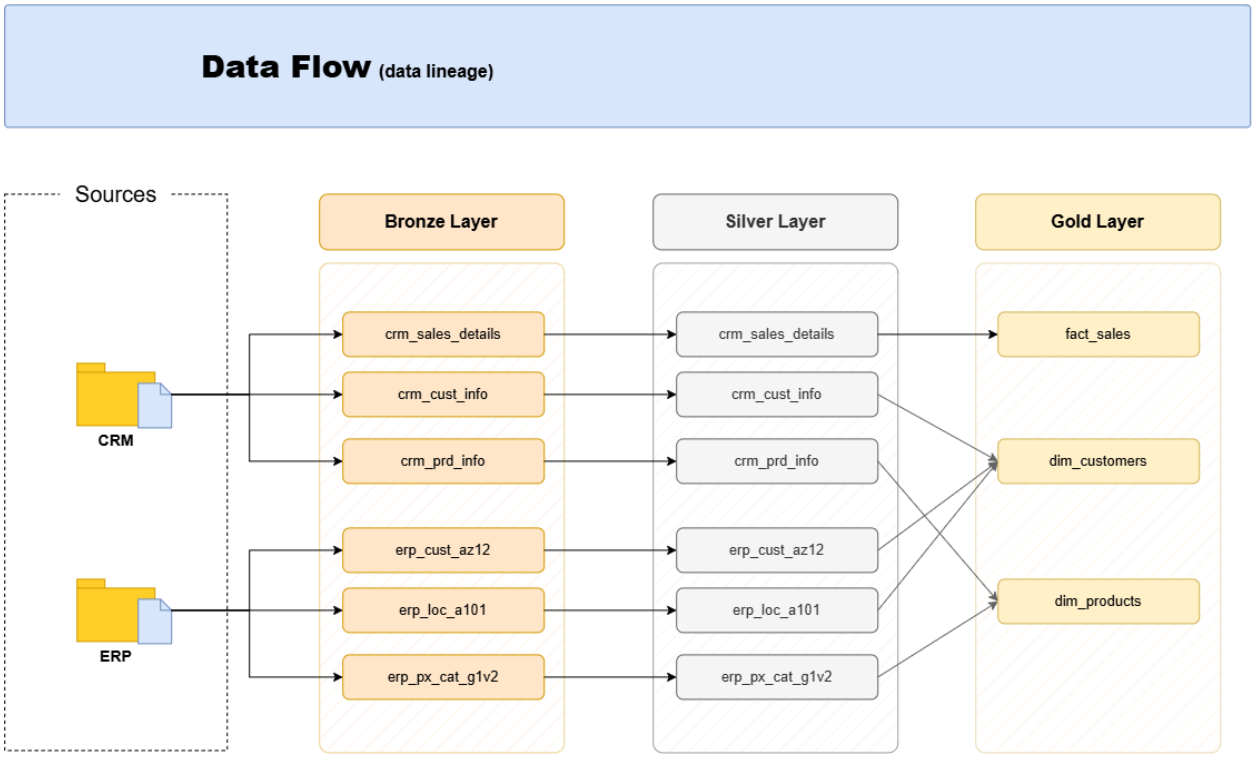
Two source systems provide six CSV files:

System	File	Content
CRM	`cust_info.csv`	Customer profiles
CRM	`prd_info.csv`	Product details and history
CRM	`sales_details.csv`	Sales transactions
ERP	`cust_az12.csv`	Extra customer data (birthdate, gender)
ERP	`loc_a101.csv`	Customer country
ERP	`px_cat_g1v2.csv`	Product categories

The CRM sales table links to products and customers through the product key and customer ID. The ERP tables connect to the CRM customer and product keys, which is how the two systems are combined.

---

3. Data flow



Each source file is loaded into a Bronze table, cleaned into a Silver table, and then combined into the Gold views.

---

4. What each layer does

Bronze: raw data

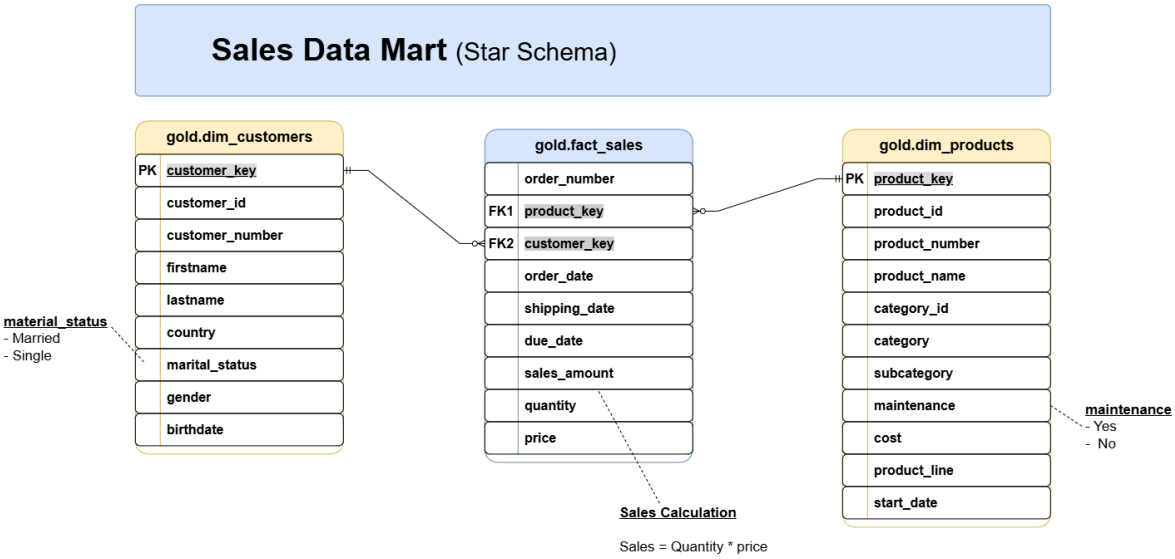
- Loads the six CSV files with `BULK INSERT`.
- Applies no transformations.
- Runs through the stored procedure `bronze.load_bronze`, which truncates each table first, logs how long each load takes, and reports errors with `TRY...CATCH`.

Silver: cleaned and standardized data

The stored procedure `silver.load_silver` reloads the layer and applies these transformations:

- **Duplicates:** keeps only the latest record per customer.
- **Text cleanup:** trims extra spaces from names and product names.
- **Standardization:** decodes codes into readable values (gender, marital status, product line, country).
- **Missing values:** replaces missing values with defaults (`n/a`, or 0 for missing costs).
- **Dates:** converts integer dates (`YYYYMMDD`) to real `DATE` values, and turns invalid dates and future birthdates into `NULL`.
- **Derived columns:** builds product end dates from the next version's start date, and recalculates sales and price when they are missing or inconsistent.
- **Key alignment:** removes prefixes and hyphens from ERP customer IDs so they match the CRM customer key.

Gold: star schema



Three views are built on top of Silver:

- `gold.dim_customers`: customers from CRM combined with ERP birthdate, gender and country.
- `gold.dim_products`: current products from CRM combined with ERP categories.
- `gold.fact_sales`: sales transactions linked to both dimensions through surrogate keys.

---

5. Data model

The Gold layer is a star schema (sales data mart). `gold.fact_sales` holds the measures (sales amount, quantity, price) and links to the dimensions through `customer_key` and `product_key`. Sales amount is calculated as quantity multiplied by price.

---

6. Repository structure

sql-data-warehouse-project/

├── datasets/

│ ├── source_crm/ # cust_info, prd_info, sales_details (CSV)

│ └── source_erp/ # cust_az12, loc_a101, px_cat_g1v2 (CSV)

├── docs/ # documentation and diagrams

├── scripts/

│ ├── bronze/ # ddl_bronze.sql, proc_load_bronze.sql

│ ├── silver/ # ddl_silver.sql, proc_load_silver.sql

│ └── gold/ # ddl_gold.sql

├── LICENSE

└── README.md

---

7. How to run it

Requirements: SQL Server (2017 or later, for the `TRIM` function) and SQL Server Management Studio (SSMS).

1. **Create the database and schemas.** Create a database and three schemas named `bronze`, `silver` and `gold`.
2. **Create the Bronze tables.** Run `scripts/bronze/ddl_bronze.sql`.

3. **Set the file paths.** In `proc_load_bronze.sql`, change each `BULK INSERT` path to the location of the CSV files on your computer. The files must be reachable by the SQL Server service.
4. **Create and run the Bronze load.**

   EXEC bronze.load_bronze;

5. **Create the Silver tables, then run the Silver load.** Run `scripts/silver/ddl_silver.sql`, create the procedure from `proc_load_silver.sql`, then run:

   EXEC silver.load_silver;

6. **Create the Gold views.** Run `scripts/gold/ddl_gold.sql`.
7. **Check the result.**

   SELECT * FROM gold.dim_customers;

   SELECT * FROM gold.dim_products;

   SELECT * FROM gold.fact_sales;

Re-running the two load procedures at any time refreshes the data, because each one empties its tables before loading.

---

8. A data-quality issue I found

While building the Gold layer, an extra row appeared at the top of `gold.dim_customers`: a customer with a `NULL` ID and almost no other data. The cause was in the Silver layer: the `ROW_NUMBER()` deduplication treats all `NULL` IDs as one group, so one empty record survived. Adding `AND cst_id IS NOT NULL` to the Silver query removed it and brought the customer table back to its expected row count.

---

9. Tools used

- SQL Server and SSMS
- T-SQL (stored procedures, window functions, views)
- draw.io for the diagrams
- Git and GitHub

---

10. Acknowledgments

Thanks to Data with Baraa for the tutorial and the datasets. See the `LICENSE` file in the repository root for the license terms.